Term Project option 2: QR factorization

1. Introduction

In this project, you are asked to develop a QR factorization module. Given a N×N matrix, the QR factorization convert it to the product of a unitary matrix Q and an upper triangular matrix R. QR factorization is useful in many digital communication applications and usually demands a high computing throughput. This calls for a dedicated hardware design to meet the demands.

2. QR factorization scheme

Assume matrix $\mathbf{A}_{4\times4}$, you may apply a sequence of Givens rotation to convert it into an upper triangular matrix $\mathbf{R}_{4\times4}$. However, to obtain the $\mathbf{Q}_{5\times5}$, you need extra computations to obtain the product of these Givens rotations. An easy way to accomplish it is augmenting matrix $\mathbf{A}_{4\times4}$ with an identity matrix $\mathbf{I}_{4\times4}$ and updating the identity matrix along with every Givens rotations applied to matrix $\mathbf{A}_{4\times4}$.

$$\mathbf{Q_n}\times\mathbf{Q_{n-1}}\times\cdots\times\mathbf{Q_2}\times\mathbf{Q_1}\times[\mathbf{A}\,|\,\mathbf{I}]=[\mathbf{R}\,|\,\mathbf{Q}]$$
$$\mathbf{Q}=\mathbf{Q_n}\times\mathbf{Q_{n-1}}\times\cdots\times\mathbf{Q_2}\times\mathbf{Q_1}$$

In this project, you are asked to develop a QR factorizer based on the triangular systolic array structure shown in the lecture note (Fig. 2).

Note that this design computes the **R** matrix only but not the **Q** matrix. Therefore, an additional rectangular array is needed to serve the purpose. This makes the array structure of the entire design a trapezoidal one (Fig. 1).

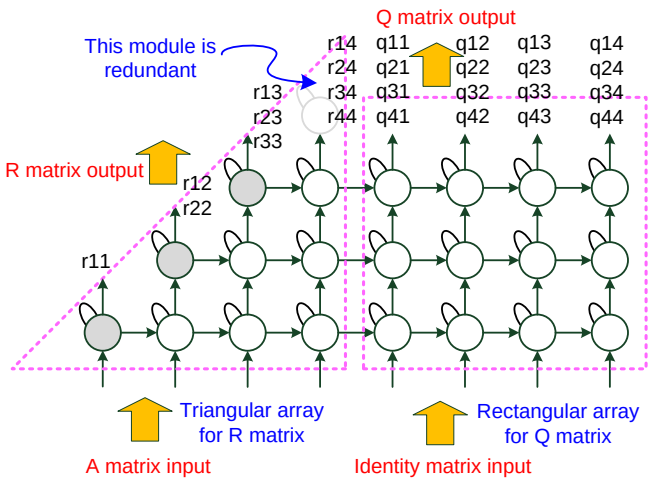


Figure 1. Trapezoidal array for QR factorization

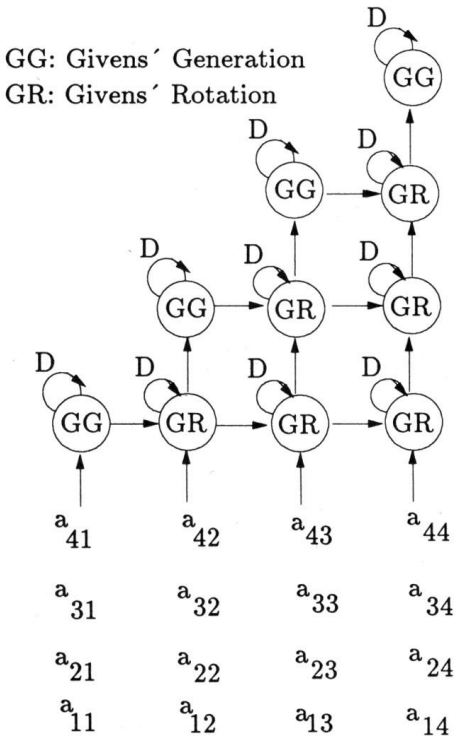


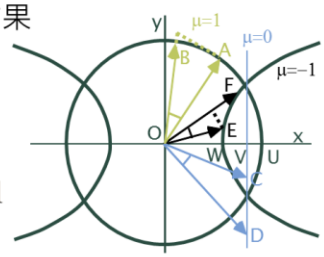
Figure 2 Tri-array for R matrix

3. Basics of CORDIC

Each module performs a Givens rotation and can be implemented using a CORDIC processor. A CORDIC computing scheme is an iterative algorithm to compute various arithmetic functions. There are three types of CORDIC computations, i.e., circular, linear and hyperbolic. Type 1 (circular) is specific to trigonometric functions. In each iteration, it needs only shift and add operations and thus hardware economical in implementation. Circular type CORDIC is further divided into vectoring mode and rotation mode. The modules indicated as GG (or the gray colored ones) performs in the vectoring mode while all other modules perform in the rotation mode.

❖ 座標軸數位旋轉計算器(Coordinate Rotation Digital Computer , CORDIC)

- 僅需執行一連串的加減法以及移位運算，最後在乘上一個常數 ，便可得到各種常見數學運算的結果
- 硬體設計複雜度低
- 環形類型(Circular type) $\xi = +1$
- 線性類型(Linear type) $\xi = 0$
- 雙曲線類型(Hyperbolic type) $\xi = -1$
- Recursive equation



$$X_{i+1} = X_i - \xi \cdot \sigma_i \cdot 2^{-i} \cdot Y_i$$
$$Y_{i+1} = Y_i + \sigma_i \cdot 2^{-i} \cdot X_i \qquad \sigma_i \in \{1, -1\} \qquad \alpha = \begin{cases} \tan^{-1}(2^{-i}) \\ 2^{-i} \\ \tanh^{-1} 2^{-i} \end{cases}$$
$$Z_{i+1} = Z_i - \sigma_i \cdot \alpha$$

	Rotation mode $d_i = \text{sign}(z_i)$, $z_i \to 0$	Vectoring mode $d_i = -\text{sign}(x_i\, y_i)$, $y_i \to 0$
$\mu = 1$ Circular $e(i) = \tan^{-1} 2^{-i}$	x → CORDIC → K(x cos z − y sin z) y → CORDIC → K(y cos z + x sin z) z → CORDIC → 0 For cos & sin, set x=1/K, y=0 tan z = sin z / cos z	x → CORDIC → $K(x^2 + y^2)^{1/2}$ y → CORDIC → 0 z → CORDIC → $z + \tan^{-1}(y/x)$ For $\tan^{-1}$, set x = 1, z = 0 $\cos^{-1} w = \tan^{-1}[(1-w^2)^{1/2}/w]$ $\sin^{-1} w = \tan^{-1}[w/(1-w^2)^{1/2}]$

4. Hardware design guidelines

- **CORDIC module**

The precision of the CORDIC computations depends on the word length as well as the iteration number. Normally, a fixed point simulation is required to obtain these numbers. In this project, for simplicity, these numbers are given. The word length is set as 12 bits and you can determine how many bits are for the integral part and how many bits are for the fractional part. The iteration number is set as 12. In other words, if one iteration is performed per clock cycle, it will take 12 clock cycles to complete the computation. This will lead to a very long computing latency of the QR factorization. A folded CORDIC architecture shown below is suggested. Two iterations are computed in one stage and the

same hardware is utilized repetitively for all iterations. This can reduce the computing latency from 12 clock cycles to 6 clock cycles. Similarly, you may put more iterations into one stage to further reduce the computing latency at the expense of increased hardware complexity and a prolonged clock period. At the end of the iterations, a scaling factor is multiplied to obtain the final result. A constant multiplier can be used for this.

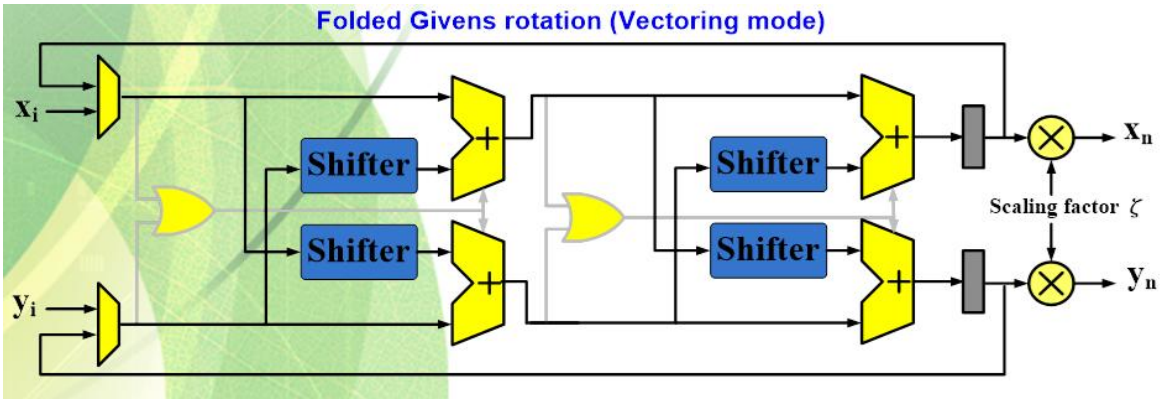


Figure 3. folded CORDIC processor design

Note that there is no need of computing the rotation angle θ explicitly. In each row of the CORDIC array, the leading one (performing in the vectoring mode) passes the rotation directions sequence, which consists of +1 or -1, to the trailing modules (performing in the rotation mode) in the same row. In other words, these trailing modules simply follow the sequence of micro-rotations performed in the leading one.

- **I/O requirements**

As shown in Figure 1, matrix A is inputted from the bottom of the triangular array and an identity matrix is inputted from the bottom of the rectangular array. The resultant upper triangular matrix R can be obtained along the diagonal border of the triangular array. The Q matrix will reside on the rectangular array. Four additional clock cycles are needed to propagate the results upward so that the Q matrix can be obtained from the top row of the rectangular array. Also assume the entries of input matrix A are all integral and 8-bit wide. The entries of R and Q matrices are all 12-bit wide.

4. Design Merits

You are asked to develop a systolic array design of the QR factorizer as indicated in Figure 1. Not shown in the figure includes a controller module (or FSM) to control the computations. Please write the corresponding Verilog code and verify the correctness of the design through simulations. The design is then synthesized to obtain the hardware features. There are two target technologies for synthesis to choose

A. FPGA (for undergraduates only, choose any Xilinx FPGA device available as the target)
B. ADFP Cell based design using Synopsys Design Compiler (choose any process technology available as the target)

There are three levels of accomplishment.

- Level 1 (minimum requirement) for undergraduate students
  Matrix A is real-valued. Only matrix R is calculated.
- Level 2 for graduate students
  Matrix A is real-valued. Both R and Q matrices are calculated.

The effectiveness of your design will be evaluated based on the following performance indexes:

- Total logic gate count – for the data path and the control path
- Maximum working frequency – in the unit of MHz
- Clock cycles needed to complete one QR factorization
- Throughput rate – number of factorization can be processed per second
- Power consumption at maximum working frequency (if synthesis result is available)
- Energy needed to complete the prediction of one image = power/throughput rate (if synthesis result is available)

5. Hand in requirements

- A report in MS word format highlights your architecture design (system block diagram), data path design, memory configuration, simulation waveforms and synthesis results, and the performance indexes stated above. You are also suggested to describe the algorithm mapping techniques applied in this project.
- A power point file (no more than 5 pages) to give an overview of your design.
- A tar file containing all design files and documents for the uploading purpose.